Institute for Cognitive Systems
Technische Universität München
Prof. Gordon Cheng

Humanoid Sensors and Actuators
Tutorial 2 - Part 2

Course Instructor: Dr. Florian Bergner
hsa-lecture.ics@xcit.tum.de

Summer Semester 2026

Microcontrollers: ADC, PWM, and Servo Motors (132 points)

In the second part of tutorial 2 we will have a look at the analog-digital comparator (ADC) peripheral block of the microcontroller. The ADC allows us to sample analog signals, thus to embed the microcontroller in analog circuits. We will also look into Timer/Counters, PWM generation, and servo motors.
In this tutorial we will learn:

- How to use the ADC peripheral block
- How to sample analog signals and send them to the computer
- How to measure the capacitance in a RC serial circuit
- How to use the Timer/Counters to generate interrupts
- How to generate Pulse Width Modulation (PWM) signals

Initial Setup (Before the Tutorial!)

Pull the updated Docker image

You need to pull the updated Docker image. The docker image you pulled in tutorial session 1 part 2 is outdated.

```
# Pull image
docker login "gitlab.lrz.de:5005"
docker pull "gitlab.lrz.de:5005/hsa/students/docker/avr/avr:focal-vscode"
docker tag "gitlab.lrz.de:5005/hsa/students/docker/avr/avr:focal-vscode" \
"avr:focal-vscode"
```

Clone the tutorial project

```
git clone "https://gitlab.lrz.de/hsa/students/hsa_t2s2_ws.git"
```

Open the tutorial project in the Dev Container

1. Remove any previously started Dev Containers of the project.

```
docker rm hsa_t2s2_ws_devcont
```

2. Open the tutorial project in VS Code:

```
cd hsa_t2s2_ws
code .
```

3. Press *Ctrl+Shift+P*, type `Dev Containers: Rebuild and Reopen In Container`, and press *Enter*. The project is now opened in the Dev Container and all terminals in VS Code will be running in the container environment.

4. After you built the Dev Container you can also open it later again with `Dev Containers: Reopen In Container` and skip the container building process.

6 Using the ADC Peripheral Block (49 points)

6.1 Testing the ADC peripheral block (3 points)

Program your microcontroller with the file `adc_uart.hex`:

```
cd hsa_t2s2_ws
cd hex
# flash the program uart_hello_world.hex
avrdude -c avrispmkII -P usb B10 -p atmega32 -U flash:w:adc_uart.hex
```

Connect the FTDI device to the UART port of the microcontroller. Check the schematic provided in tutorial 1 part 1.
Then launch the Python script that connects to the FTDI device and observe the printout:

```
cd hsa_21s2_ws
cd src/adc_poti
# run the python script, you can exit it by pressing 'q' and then 'Enter'.
##    use 's' and then press 'Enter' to start the logging and print out
##    use 'e' and then press 'Enter' to stop the logging and print out
./log.py
```

T.6.1 (1 point) How does a potentiometer work? What are the pinouts and how should it be connected?

1. Connect a 100kΩ potentiometer. Connect one of the outer pins to GND and the other to 5V. Connect the middle pin to ADC0 (see µC pinout of T1.1).

T.6.2 (1 point) Explain what you see in the output when you change the value of the potentiometer? What are the minimum and maximum values?

T.6.2 (1 point) What is this microcontroller program doing?

T.6.3 (Bonus: 5 points) Use the `log.py` Python script as a template and write a new script that opens the serial port and plots the reading of the ADC in real-time. Submit the new script `adc_plotter.py`.

6.2 Sampling Analog Signals with the ADC (18 points)

Please use the tutorial project `hsa_t2s2_ws` as basis for the tasks introduced in this section. Please submit the code you created as specified in the tasks. You can find template files for each task in the folder `hsa_t2s2_ws/src/adc_poti/src/applications`.

T.6.4 **(8 points)** Consult the data sheet `Atmega32.pdf` (*hsa_t2s2_ws/docs/*) of the AVR Atmega32. Implement your own program in C to sample analog signals with the ADC considering the following instructions to set the proper bits in the registers:

- **(2 points)** The ADC uses the AVCC voltage (5 V) and assume that an external decoupling capacitor is connected to the AREF pin.
- **(2 points)** The ADC uses a prescaler of 2 such that the operating frequency of the ADC is half the CPU frequency, in our case 500 kHz.
- **(2 points)** The ADC operates in the free running mode.
- **(2 points)** The ADC operates in the 8 bit sampling mode, i.e. the correct 8 bits of a 8 bit sample are loaded into one sample register of the ADC.

T.6.5 **(2 points)** Use the AVR UART library introduced in T1.3 as provided in the template file `hsa_t2s2_ws/src/adc_poti/src/applications/main_adc_poti.c` of project to communicate with the computer.

T.6.6 **(4 points)** Implement a function to sample and read the 8bit ADC value for a specified channel using the following function declaration:

```
void adc_readBlocking(uint8_t* b, uint8_t ch);
```

- **(2 points)** The function uses the `ch` argument and samples the correct channel.
- **(2 points)** The function samples and returns the correct 8 bits of the sample.

T.6.7 **(2 points)** Send the samples of pin `ADC2` in binary format to the computer.

T.6.8 **(2 points)** Connect the potentiometer to the `ADC2` pin, such that you can use the potentiometer to set voltages between 0V and 5V. Use the Python scripts `log.py` and `plot.py` and capture the data while turning the potentiometer. The computer program generates a CSV file `adc_log.csv`. You can plot this file with the Python script `plot.py`.

Please submit the following files containing your solution for tasks **T.6.4** to **T.6.8**:

- `main_adc_poti.c`
- `adc_poti.csv` containing the sample of your potentiometer experiment
- A screenshot `adc_poti.png` of the plot that you created with the `adc_poti.csv`

6.3 Measuring the Capacitance of a Capacitor with the ADC (14 points)

Now, we use a simple RC series circuit to measure the capacitance of a capacitor with the ADC. While we want to determine the capacitance, the resistance and the sampling frequency are known.
Please submit the code you created as specified in the tasks. You can find template files for each task in the folder `hsa_t2s2_ws/src/adc_rcsc/src/applications`.
The template files of this section use pre-compiled libraries for the UART and the ADC peripheral blocks. Please use these libraries for implementing the tasks of this section. You can implement all tasks of this section without completing the tasks of the previous sections.

T.6.9 **(2 points)** Use the pin `PC1` to charge/discharge a 1 µF capacitor via a 1kΩ resistor.

T.6.10 **(4 points)** Visualize in the oscilloscope the charge/discharge process.

T.6.11 **(4 points)** Use the `ADC0` pin to sample the loading curve with maximum ADC sampling frequency. To minimize any delays between samples first store 1024 samples into the SRAM.

T.6.12 **(2 points)** Then send the 1024 samples you stored into the SRAM to the computer using the

```
void uart_writeBlocking(const uint8_t* d, uint8_t size);
```

function.

T.6.13 **(2 points)** Save the CSV file of the 1024 samples and plot it. Note: You will need this CSV file to answer some questions in your report.

Please submit the following files containing your solution for tasks **T.6.9** to **T.6.13**:

- `main_adc_rcsc.c`
- `adc_rcsc.csv` containing the samples of your RC series circuit experiment
- A screenshot `adc_rcsc.png` of the plot that you created with the `adc_rcsc.csv`
- A picture `rc_scope.png` of your oscilloscope measurement

6.4 Report (14 points)

R.6.1 **(4 points)** In **T.6.4** you use the ADC with specific settings and a CPU frequency of 1 MHz. How many CPU cycles pass between the acquisitions of two ADC samples? Elaborate and explain.

R.6.2 **(2 points)** What is then the sampling frequency of the ADC?

R.6.3 **(2 points)** Assuming that each iteration of your for-loop for the 1024 samples (**T.6.11**) takes 32 µs, what is limiting your actual sampling frequency:

- The iteration time of the for-loop (32 µs), or
- the sampling time of the ADC in free running mode?

R.6.4 **(2 points)** Use your results of **T.6.13** and measure the 3τ ($\tau = RC$) value. Remember the capacitance is not known and will be determined using the τ value.

R.6.5 **(2 points)** What ADC tick value (range: 0 to 255) did you use for measuring 3τ? Please explain your computations in detail.

R.6.6 **(2 points)** What voltage level does this ADC value represent? Please explain your computations in detail.

R.6.7 **Bonus (2 points)** Use the τ value you determined in **R.6.4** and the given resistance $R = 1k\Omega$ and calculate the capacitance C of the capacitor.

R.6.8 **Bonus (4 points)** We discussed the operation of the ADC in the lecture. Let's assume that the conversion logic uses a digital counter with a DAC to generate a voltage ramp. Considering the Sample&Hold time (see datasheet) and the total conversion time of the ADC (see datasheet, **R.6.1**) with the settings used in **T.6.4**, what would be the minimal frequency of the clock that drives the digital counter generating that voltage ramp?

7 AVR Timers/Counters (25 points)

A timer is a special peripheral register whose value is incremented or decremented automatically by the microcontroller at a user-defined multiple of the clock frequency. A 8-bit timer for example, has values ranging from 0 to (2^8 - 1) = 255 and will therefore reach its maximum value in *K* x 255 clock ticks, where *K* is the so called timer *clock prescaler*. When a timer reaches its maximum value, the next *K*-multiple clock tick makes it "*overflow*". This means that its register value is rolled back to 0 and that a new count cycle is automatically initiated. When a timer overflows or reaches a predefined value, an interrupt can be generated to inform the CPU and potentially triggers some actions at a well defined rate. The AVR has 3 different timers: One of 16 bits and two of 8 bits.

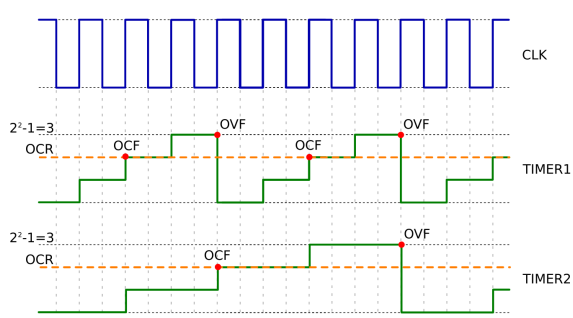

Figure 1: Timing Diagrams of two hypothetical 2-bit timers with values ranging from 0 to 3, namely TIMER1 with a clock prescaler *K* = 1 and TIMER2 with *K* = 2, alongside with the clock signal CLK of the microcontroller. The overflow flag and output-compare flags are here labeled as OVF and OCF respectively. Note that more detailed versions of this diagram are available on the AVR datasheet at pages 105 and 106.

Setup

- Please submit the code you created as specified in the tasks. You can find template files for each task in the folder `hsa_t2s2_ws/src/timer/src/applications`.
- In this first exercise, we want to use the overflow interrupt of TIMER2 (8-bits) in order to blink a LED connected to the pin PC0 of the microcontroller, at exactly 0.5Hz. Please connect your LED with an appropriate resistance to PC0 as in the past tutorials.

7.1 Code (10 points)

T.7.1 (5 points) Within your "`main()`" function of `main_timer_blink.c`:

- Start by including the AVR interrupt library within your code template: `#include <avr/interrupt.h>`
- You should then disable the interrupts[1]. This is in general a good practice, which prevents the microcontroller from being interrupted while performing important operations.
- In the `TIMER2 Control Register` (TCCR2), initialize the timer clock prescaler to 64[2].
- Set TIMER2 to issue an interrupt when an overflow event is detected. To do so, set the "`Overflow Interrupt Enable`" in the `Timer Interrupt Mask Register` (TIMSK)[3].
- Initialize the `TIMER2 Counter Register` (TCNT2) to 0. The value of this register will be automatically incremented by one unit at each tic of the prescaled clock, and will roll back to zero at overflow.
- Set C0 as the desired output pin.
- You can now re-enable the interrupts by setting the global interrupt enable.
- Define a dummy infinite loop in order to maintain your microcontroller active.

T.7.2 (5 points) Outside the "`main()`" function:

- Write an *interrupt service routine (ISR)* catching the overflow interrupt vector of TIMER2:

```
ISR(TIMER2_OVF_vect)
{
...
}
```

- Use a *global variable*[4] in the ISR, so that the LED state changes after a defined number of counter overflow, corresponding to a blinking frequency of 0.5 Hz. Give this number into your report and justify.

Please submit the following files:

- `main_timer_blink.c`

[1] ***Hint:*** take a look in `avr/interrupt.h`
[2] ***Hint:*** take a look p. 125-127 of the AVR manual.
[3] ***Hint:*** take a look p. 130 of the AVR manual.
[4] You should use the prefix `volatile` outside the main.

7.2 Report (15 points)

R.7.1 (3 points) Define the following terms using your own words:

1. Interrupt
2. Interrupt service routine
3. Busy wait state

R.7.2 (2 points) Explain why interrupts are so appealing compared to busy wait states.

R.7.3 (5 points) What happens when an interrupt flag is raised during the execution of another interrupt routine? Justify by referring to the relevant datasheet page number(s).

R.7.4 Bonus (3 points) Based on your answer to the previous question, discuss whether an external interrupt request, say `INT0`, can interrupt the ISR of `TIMER2 OVF`.

R.7.5 (5 points) What happens on the AVR when an ISR takes too long to execute? Is it then possible to interrupt an ISR with the same interrupt flag that triggered it in the first place? Justify[5].

R.7.6 Bonus (5 points) Knowing the clock frequency of the microcontroller and the size of the timer register, propose a new timer clock prescaler value, allowing to get as close as possible to the desired blinking frequency with the previously implemented method. Justify.

[5] Note that although it is a nice complement, a screen capture alone cannot be considered as a full justification. Moreover, never forget that google is your friend.

8 Pulse Width Modulation (20 points)

Digital devices can only generate two voltage levels: HIGH=5 V and LOW=0 V. Pulse Width Modulation (PWM) is a widely used modulation technique which consists in varying the portion of the time a periodic digital signal spends in state "HIGH" versus the time that this signal spends in state "LOW". In this context, we define the so called *Duty cycle* of such a signal as:

$$D = \frac{T_{\text{HIGH}}}{T_{\text{HIGH}} + T_{\text{LOW}}} 100\% \tag{1}$$

PWM signals with different frequencies and duty cycles can be easily generated by exploiting some specific features of timers, namely the prescaler, the overflow interrupt and the compare-match interrupt. When a timer is properly configured (c.f. AVR datasheet), its value is incremented until it matches that of the corresponding *Output Compare Register (OCR)*, previously defined by the user. In general, at this stage, there are two possibilities to generate a PWM on an AVR:

1. Either by playing with the overflow and output-compare ISR (c.f. AVR datasheet, p.71)
2. or by using the Compare-Match-Output Unit in wave-generation mode (c.f. AVR datasheet, p.72)

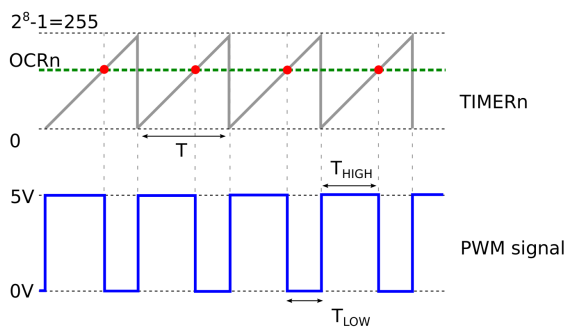

Figure 2: PWM generation principle on a microcontroller: The PWM signal (in blue) is here obtained with a 8-bit timer (`TIMERn` in gray) and output compare register `OCRn` (in dashed green).

During this tutorial session, we will explore the first PWM generation mode and play with ISR.

8.1 Code (15 points)

Please submit the code you created as specified in the tasks. You can find template files for each task in the folder `hsa_t2s2_ws/src/timer/src/applications`.

T.8.1 (5 points) PWM generation with fixed duty cycle, modify `main_timer_pwm.c`:

- Start by including the AVR interrupt library within your code template: `#include <avr/interrupt.h>`
- Initialize the TIMER0 clock prescaler to 1024.
- Set TIMER0 to issue an interrupt when an overflow event is detected. To do so, set the "`Overflow Interrupt Enable`" in the `Timer Interrupt Mask Register` (TIMSK).

- Set TIMER0 to issue an interrupt when a compare-match event is detected. To do so, set the "`Output Compare Interrupt Enable`" in the `Timer Interrupt Mask Register` (TIMSK).
- Set C0 as the desired output pin.
- Define a dummy infinite loop in order to maintain your microcontroller active.
- Following the timing diagram of Figure 2, implement a PWM generation routine using the OVF and OCF ISR.

T.8.2 (10 points) PWM generation with adjustable duty cycle, modify `main_timer_pwm_adc.c`:

- The idea is here ot use the 8-bit values read by the ADC of your microcontroller as duty cycle reference for your PWM signal.
- Start by setting the ADC with the following settings:
  * Use the ADC in the 8 bit mode.
  * Connect the potentiometer to the ADC0 pin.
  * Use an ADC prescaler of 128 so that the duty cycle of your PWM is not changed too frequently.
  * Do not use the free running mode but rather the interrupt generation mode (c.f. AVR datasheet p.216) so that when a conversion is finished, an interrupt is issued by the ADC and processed in a dedicated ISR:

```
ISR(ADC_vect)
{
...
}
```

  Use this ISR to update a global variable with the read ADC value. You can then use this variable to update the value of the OCR register in the OVF ISR.
- Light a LED with the obtained signal. What do you observe when you change the duty cycle? What happens when the value of the duty cycle gets close to a minimum or a maximum? *Hint:* Use an oscilloscope to visualize the signal.

Please submit the following files:

- `main_timer_pwm.c`
- `main_timer_pwm_adc.c`

8.2 Report (5 points)

R.8.1 (2 points) Which AVR timers can be used for PWM generation? Why? Justify by refering to the relevant datasheet page numbers.

R.8.2 (3 points) Propose an explanation to the observations you made in **T.8.2** regarding the minimum and maximum values of the duty cycle.

R.8.3 Bonus (2 points) Explain in your own words what `Double-Buffering` is (you may take a look at the AVR dataseet).

9 Servo Motors, ADC, and Flex-Sensors (38 points)

Please submit the code you created as specified in the tasks. You can find template files for each task in the folder `hsa_t2s2_ws/src/timer/src/applications`.

T.9.1 **(10 points)** Connect a servo motor to your microcontroller. Connect the servo motor also to +5V and GND. Generate a PWM signal to drive it through its full range. Check the datasheet of a servo motor. Most servo motors use the standard protocol. Visualize the changes of the duty cycle on the oscilloscope. Please submit:

- A video of the servo motor through its full rotation range `timer_servo.mp4`
- Oscilloscope pictures of three different angle readings (indicate the period and duty cycle) `timer_servo_1.png`, `timer_servo_2.png`, `timer_servo_3.png`
- Your code in `main_timer_servo.c`

T.9.2 **(3 points)** Use a multimeter to read the resistance values of a flex sensor when extended and flexed. Describe the behavior between flection/extension and resistance?

T.9.3 **(5 points)** Build a resistor divider with the flex sensor and connect it to the pin ADC0. Calculate values for the resistor divider to measure the full range of the flex sensor. Explain your calculations and how you chose the resistor.

T.9.4 **(10 points)** Read the values of the flex sensor with your ADC and send them over UART. What are the limits? How can you tune the ADC to have the best accuracy and range the microcontroller can offer? Please submit your code in `main_timer_sensor.c`.

T.9.5 **(10 points)** Program your microcontroller to move the motor when you flex the sensor. It should go through the whole range of the motor and the flex sensor. Please submit:

- Your code in `main_timer_servo_sensor.c`
- A video `main_timer_servo_sensor.mp4`
- A short explanation how your program works